NAME : KISHORE KUMAR V

REG NO : 192521371

COURSE CODE : ITA0613

COURSE NAME : MACHINE LEARNING

FIND-S Algorithm: Finding the Most Specific Hypothesis

Implementation and demonstration of the FIND-S algorithm using five different training data sets. Each experiment contains the Aim, Algorithm, Python Code, Output and Result.

Question 1: EnjoySport Concept Learning

Problem Statement

Apply FIND-S to the EnjoySport training data to find the most specific hypothesis for the days on which Aldo enjoys his favourite water sport.

Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Warm	Normal	Strong	Warm	Same	Yes
Sunny	Warm	High	Strong	Warm	Same	Yes
Rainy	Cold	High	Strong	Warm	Change	No
Sunny	Warm	High	Strong	Cool	Change	Yes

Aim

To implement the FIND-S algorithm in Python and find the most specific hypothesis consistent with the EnjoySport training examples.

Algorithm

1. Start with the most specific hypothesis h (every attribute set to Ø / the first positive example).
2. For each training example x, check whether it is a positive example (class label = Yes).
3. If x is negative, ignore it; FIND-S does not change h for negative examples.
4. If x is positive, compare each attribute value of x with the same attribute of h.
5. If the values are equal, keep the attribute in h; otherwise replace it with the general symbol '?'.

6. Repeat for all training examples and print h after every step.
7. Output the final hypothesis h as the most specific hypothesis.

Python Code

```
# Q1: EnjoySport
attrs = ["Sky", "AirTemp", "Humidity", "Wind", "Water", "Forecast"]
data = [
 ["Sunny","Warm","Normal","Strong","Warm","Same","Yes"],
 ["Sunny","Warm","High","Strong","Warm","Same","Yes"],
 ["Rainy","Cold","High","Strong","Warm","Change","No"],
 ["Sunny","Warm","High","Strong","Cool","Change","Yes"],
]

def find_s(data):
    h = ["0"] * (len(data[0]) - 1)
    print("Initial h =", h)
    for i, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":
            for j in range(len(h)):
                if h[j] == "0":
                    h[j] = x[j]
                elif h[j] != x[j]:
```

```
                h[j] = "?"
        print(f"Instance {i} ({label}) -> h = {h}")
    return h

print("Attributes:", attrs)
final = find_s(data)
print("Final Hypothesis:", final)
```

Output

```
Attributes: ['Sky', 'AirTemp', 'Humidity', 'Wind', 'Water', 'Forecast']
Initial h = ['0', '0', '0', '0', '0', '0']
Instance 1 (Yes) -> h = ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
Instance 2 (Yes) -> h = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
Instance 3 (No) -> h = ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']
Instance 4 (Yes) -> h = ['Sunny', 'Warm', '?', 'Strong', '?', '?']
Final Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']
```

Result

The FIND-S algorithm gave the most specific hypothesis <Sunny, Warm, ?, Strong, ?, ?>. Aldo enjoys the sport when the sky is Sunny, the air is Warm and the wind is Strong.

Question 2: Weather Data (Play Tennis)

Problem Statement

Use FIND-S on the weather data set to find the most specific hypothesis describing the conditions under which tennis can be played.

Outlook	Temp	Humidity	Wind	PlayTennis
Sunny	Hot	Normal	Weak	Yes
Rain	Hot	High	Weak	No
Sunny	Mild	Normal	Weak	Yes
Overcast	Cool	High	Strong	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	High	Strong	No
Sunny	Mild	Normal	Weak	Yes

Aim

To apply the FIND-S algorithm on the weather data set and obtain the most specific hypothesis for playing tennis.

Algorithm

1. Start with the most specific hypothesis h (every attribute set to Ø / the first positive example).
2. For each training example x, check whether it is a positive example (class label = Yes).

3. If x is negative, ignore it; FIND-S does not change h for negative examples.
4. If x is positive, compare each attribute value of x with the same attribute of h.
5. If the values are equal, keep the attribute in h; otherwise replace it with the general symbol '?'.
6. Repeat for all training examples and print h after every step.
7. Output the final hypothesis h as the most specific hypothesis.

Python Code

```
# Q2: PlayTennis
data = [
 ["Sunny","Hot","Normal","Weak","Yes"],
 ["Rain","Hot","High","Weak","No"],
 ["Sunny","Mild","Normal","Weak","Yes"],
 ["Overcast","Cool","High","Strong","No"],
 ["Sunny","Cool","Normal","Weak","Yes"],
 ["Rain","Mild","High","Strong","No"],
 ["Sunny","Mild","Normal","Weak","Yes"],
]

def find_s(data):
    h = None
    for i, row in enumerate(data, 1):
        x, label = row[:-1], row[-1]
        if label == "Yes":
```

```
            if h is None:
                h = x[:]
            else:
                h = [a if a == b else "?"
                     for a, b in zip(h, x)]
        print("Row", i, label.ljust(3), "h =", h)
    return h

print("Outlook, Temp, Humidity, Wind")
print("Final:", find_s(data))
```

Output

```
Outlook, Temp, Humidity, Wind
Row 1 Yes h = ['Sunny', 'Hot', 'Normal', 'Weak']
Row 2 No  h = ['Sunny', 'Hot', 'Normal', 'Weak']
Row 3 Yes h = ['Sunny', '?', 'Normal', 'Weak']
Row 4 No  h = ['Sunny', '?', 'Normal', 'Weak']
Row 5 Yes h = ['Sunny', '?', 'Normal', 'Weak']
Row 6 No  h = ['Sunny', '?', 'Normal', 'Weak']
Row 7 Yes h = ['Sunny', '?', 'Normal', 'Weak']
Final: ['Sunny', '?', 'Normal', 'Weak']
```

Result

The most specific hypothesis found is <Sunny, ?, Normal, Weak>. Tennis is played when the outlook is Sunny, humidity is Normal and wind is Weak; temperature does not matter.

Question 3: Buys Computer Prediction

Problem Statement

From the customer data below, use FIND-S to find the most specific hypothesis describing the customers who buy a computer.

Age	Income	Student	Credit	BuysComputer
Youth	Medium	Yes	Fair	Yes
Senior	High	No	Fair	No
Youth	Low	Yes	Fair	Yes
Middle	Medium	No	Excellent	No
Youth	Medium	Yes	Excellent	Yes
Senior	Low	No	Fair	No
Youth	Low	Yes	Excellent	Yes

Aim

To implement the FIND-S algorithm to find the most specific hypothesis for customers who buy a computer.

Algorithm

1. Start with the most specific hypothesis h (every attribute set to Ø / the first positive example).
2. For each training example x, check whether it is a positive example (class label = Yes).

3. If x is negative, ignore it; FIND-S does not change h for negative examples.
4. If x is positive, compare each attribute value of x with the same attribute of h.
5. If the values are equal, keep the attribute in h; otherwise replace it with the general symbol '?'.
6. Repeat for all training examples and print h after every step.
7. Output the final hypothesis h as the most specific hypothesis.

Python Code

```
# Q3: Buys Computer
data = [
 ("Youth","Medium","Yes","Fair","Yes"),
 ("Senior","High","No","Fair","No"),
 ("Youth","Low","Yes","Fair","Yes"),
 ("Middle","Medium","No","Excellent","No"),
 ("Youth","Medium","Yes","Excellent","Yes"),
 ("Senior","Low","No","Fair","No"),
 ("Youth","Low","Yes","Excellent","Yes"),
]
names = ["Age","Income","Student","Credit"]

hyp = []
for n, rec in enumerate(data, 1):
    features, buys = list(rec[:-1]), rec[-1]
    if buys != "Yes":
```

```
        print(f"Record {n}: negative, skipped")
        continue
    if not hyp:
        hyp = features
    else:
        for k in range(len(hyp)):
            if hyp[k] != features[k]:
                hyp[k] = "?"
    print(f"Record {n}: positive, h = {hyp}")

print("\nMost specific hypothesis:")
for nm, v in zip(names, hyp):
    print(f"  {nm:8s}: {v}")
```

Output

```
Record 1: positive, h = ['Youth', 'Medium', 'Yes', 'Fair']
Record 2: negative, skipped
Record 3: positive, h = ['Youth', '?', 'Yes', 'Fair']
Record 4: negative, skipped
Record 5: positive, h = ['Youth', '?', 'Yes', '?']
Record 6: negative, skipped
Record 7: positive, h = ['Youth', '?', 'Yes', '?']
```

Most specific hypothesis:
Age : Youth
Income : ?
Student : Yes
Credit : ?

Result

The final hypothesis is <Youth, ?, Yes, ?>. A customer buys a computer if he or she is a Youth and a Student, irrespective of income and credit rating.

Question 4: Loan Approval (Reading CSV Data)

Problem Statement

Read the loan applicant records in CSV format and use FIND-S to find the most specific hypothesis for approved loans.

Income	Job	Credit	Property	Approved
High	Salaried	Good	Own	Yes
High	Salaried	Good	Rent	Yes
Low	Unemployed	Poor	Rent	No
High	Salaried	Good	Own	Yes
Medium	Business	Good	Own	No
High	Salaried	Good	Own	Yes

Aim

To read a CSV data set and find the most specific hypothesis for loan approval using the FIND-S algorithm, starting from the null hypothesis.

Algorithm

1. Start with the most specific hypothesis h (every attribute set to Ø / the first positive example).
2. For each training example x, check whether it is a positive example (class label = Yes).
3. If x is negative, ignore it; FIND-S does not change h for negative examples.

4. If x is positive, compare each attribute value of x with the same attribute of h.
5. If the values are equal, keep the attribute in h; otherwise replace it with the general symbol '?'.
6. Repeat for all training examples and print h after every step.
7. Output the final hypothesis h as the most specific hypothesis.

Python Code

```
# Q4: Loan Approval (reads CSV text)
import csv, io

text = """Income,Job,Credit,Property,Approved
High,Salaried,Good,Own,Yes
High,Salaried,Good,Rent,Yes
Low,Unemployed,Poor,Rent,No
High,Salaried,Good,Own,Yes
Medium,Business,Good,Own,No
High,Salaried,Good,Own,Yes
"""

rows = list(csv.reader(io.StringIO(text)))
header, rows = rows[0], rows[1:]
hyp = ["Ø"] * (len(header) - 1)

print("Start:", hyp)
```

```
for i, r in enumerate(rows, 1):
    x, y = r[:-1], r[-1]
    if y == "Yes":
        for j, val in enumerate(x):
            if hyp[j] == "Ø":
                hyp[j] = val
            elif hyp[j] != val:
                hyp[j] = "?"
    print(f"After row {i} [{y:3s}]:", hyp)

print("\nFinal specific hypothesis:", hyp)
```

Output

```
Start: ['Ø', 'Ø', 'Ø', 'Ø']
After row 1 [Yes]: ['High', 'Salaried', 'Good', 'Own']
After row 2 [Yes]: ['High', 'Salaried', 'Good', '?']
After row 3 [No ]: ['High', 'Salaried', 'Good', '?']
After row 4 [Yes]: ['High', 'Salaried', 'Good', '?']
After row 5 [No ]: ['High', 'Salaried', 'Good', '?']
After row 6 [Yes]: ['High', 'Salaried', 'Good', '?']

Final specific hypothesis: ['High', 'Salaried', 'Good', '?']
```

Result

The most specific hypothesis is <High, Salaried, Good, ?>. A loan is approved for applicants with High income, a Salaried job and Good credit; owning or renting property is not important.

Question 5: Medical Diagnosis (Flu) using a Class

Problem Statement

Implement FIND-S as a Python class (fit and predict methods) to learn the symptoms of Flu and then test it on new patients.

Fever	Cough	BodyTemp	Aches	Flu
Yes	Yes	High	Yes	Yes
Yes	No	High	Yes	Yes
No	No	Normal	No	No
Yes	Yes	High	No	Yes
Yes	Yes	Normal	No	No

Aim

To build an object-oriented FIND-S learner in Python that finds the most specific hypothesis for Flu and uses it to classify new patients.

Algorithm

1. Start with the most specific hypothesis h (every attribute set to Ø / the first positive example).
2. For each training example x, check whether it is a positive example (class label = Yes).
3. If x is negative, ignore it; FIND-S does not change h for negative examples.
4. If x is positive, compare each attribute value of x with the same attribute of h.

5. If the values are equal, keep the attribute in h; otherwise replace it with the general symbol '?'.
6. Repeat for all training examples and print h after every step.
7. Output the final hypothesis h as the most specific hypothesis.

Python Code

```
# Q5: Medical diagnosis - Flu
class FindS:
    def __init__(self, n):
        self.h = None
        self.n = n
        self.history = []

    def fit(self, X, y):
        for xi, yi in zip(X, y):
            if yi == 1:
                if self.h is None:
                    self.h = list(xi)
                else:
                    for k in range(self.n):
                        if self.h[k] != xi[k]:
                            self.h[k] = "?"
            self.history.append(list(self.h or []))
        return self
```

```
    def predict(self, x):
        ok = all(a == "?" or a == b
                 for a, b in zip(self.h, x))
        return "Flu" if ok else "No Flu"

X = [
 ["Yes","Yes","High","Yes"],
 ["Yes","No","High","Yes"],
 ["No","No","Normal","No"],
 ["Yes","Yes","High","No"],
 ["Yes","Yes","Normal","No"],
]
y = [1, 1, 0, 1, 0]
print("Symptoms: Fever, Cough, BodyTemp, Aches")

model = FindS(4).fit(X, y)
for i, s in enumerate(model.history, 1):
    print("Step", i, "->", s)
print("Final h:", model.h)
print("Test 1:", model.predict(["Yes","Yes","High","Yes"]))
print("Test 2:", model.predict(["No","No","Normal","No"]))
```

Output

```
Symptoms: Fever, Cough, BodyTemp, Aches
Step 1 -> ['Yes', 'Yes', 'High', 'Yes']
Step 2 -> ['Yes', '?', 'High', 'Yes']
Step 3 -> ['Yes', '?', 'High', 'Yes']
Step 4 -> ['Yes', '?', 'High', '?']
Step 5 -> ['Yes', '?', 'High', '?']
Final h: ['Yes', '?', 'High', '?']
Test 1: Flu
Test 2: No Flu
```

Result

The learned hypothesis is <Yes, ?, High, ?>. A patient with Fever and High body temperature is classified as Flu; the test patient with no symptoms was correctly classified as No Flu.